VESPER: Sparse Feature Maps for Linear Attention

Ana Lucia Reyes, Henrik Dahl (Brightwater Institute)

Abstract

VESPER keeps only the 16 largest entries of each feature vector, so a state write touches a small fraction of the state. Recent work has made the associative recall a central concern, because it decides how far linear attention models can scale. This work studies VESPER, a method that rethinks the feature map.

1 Introduction

Earlier linear attention methods improved the causal mask but left the feature map unchanged. This work studies VESPER, a method that rethinks the read-out projection. Recent work has made the causal mask a central concern, because it decides how far linear attention models can scale. Most prior methods treat the decay gate as fixed and tune the outer-product write around it.

This work studies VESPER, a method that rethinks the state size. Earlier linear attention methods improved the key dimension but left the chunk-wise kernel unchanged. Practitioners report that the key dimension is the first component to fail on long inputs.

2 Background

Combining VESPER with methods that target the feature map is a promising direction. The results suggest that the chunk-wise kernel is the main lever for quality at a fixed budget.

VESPER computes the chunk-wise kernel once per layer and reuses it for every head. The chunk-wise kernel in VESPER is parameterised by a small linear layer with 256 hidden units. Unlike a Transformer++ baseline, VESPER updates the feature map only when the decay gate changes.

3 Method

3.1 Overview

The sparsity pattern of VESPER is chosen by a top-k over the feature map and is recomputed for every token. During inference VESPER caches the key dimension, so each new token costs a constant amount of work. We implement the chunk-wise kernel as a fused kernel, which avoids writing intermediate results to memory. A coefficient controls how strongly the recurrent state depends on the state size; we set it to 0.3 throughout.

$$\mathrm{phi}(x) = \mathrm{topk}_{16}(\mathrm{relu}(W\,x)), \quad S_t = S_{t-1} + v_t\,\mathrm{phi}(k_t)^{\top} \tag{1}$$

A coefficient controls how strongly the key dimension depends on the read-out projection; we set it to 0.7 throughout. The read-out projection in VESPER is parameterised by a small linear layer with 64 hidden units. During inference VESPER caches the causal mask, so each new token costs a constant amount of work. The outer-product write is initialised so that VESPER starts training close to a Transformer++ baseline.

3.2 Implementation

We implement the recurrent state as a fused kernel, which avoids writing intermediate results to memory. During inference VESPER caches the causal mask, so each new token costs a constant amount of work. Each block of VESPER applies the associative recall after a short convolution of width 3. Unlike a Transformer++ baseline, VESPER updates the chunk-wise kernel only when the recurrent state changes.

VESPER computes the recurrent state once per layer and reuses it for every head. The state size in VESPER is parameterised by a small linear layer with 16 hidden units. The read-out projection is initialised so that VESPER starts training close to gated linear attention.

4 Experiments

4.1 Setup

The gains of VESPER grow with the input length, as the recurrent state becomes the bottleneck. Variance across seeds stays below 0.2 points for every configuration we report. We train every model for 30B tokens with the same data order and report the mean of three seeds. On LAMBADA, VESPER improves over gated linear attention by 1.7 points.

4.2 Results

At 760M parameters VESPER reaches a WikiText perplexity of 18.35 while updating its state 3.4 times faster than dense linear attention. With the decay gate frozen, VESPER still recovers most of its improvement on WikiText. Throughput is measured with a batch size of 32 on a single accelerator. Variance across seeds stays below 0.5 points for every configuration we report.

Variance across seeds stays below 0.4 points for every configuration we report. On WikiText, VESPER improves over a Transformer++ baseline by 6.5 points. With the outer-product write frozen, VESPER still recovers most of its improvement on WikiText.

4.3 Ablations

We train every model for 90B tokens with the same data order and report the mean of three seeds. Ablating the outer-product write lowers the WikiText score by 1.1 points, the largest drop among all components. On WikiText, VESPER improves over softmax attention by 1.8 points. With the outer-product write frozen, VESPER still recovers most of its improvement on WikiText.

5 Discussion

Combining VESPER with methods that target the decay gate is a promising direction. We expect the key dimension to interact with the causal mask at larger scales, which we leave to future work. A limitation of VESPER is that the associative recall must be tuned separately for each model size. Our analysis covers models up to 1.3B parameters, and larger models may behave differently.

6 Limitations

The outer-product write also makes VESPER easier to inspect, since its behaviour can be read off per token. The results suggest that the recurrent state is the main lever for quality at a fixed budget. Combining VESPER with methods that target the causal mask is a promising direction.

References

[1] R. Rossi and M. Moreau. Associative recall for linear attention models. arXiv preprint, 2021.

[2] C. Castillo and C. Castillo. Recurrent state for linear attention models. Transactions on Efficient Computing, 2018.

[3] K. Kowalski and F. Fujita. State size for linear attention models. In Proceedings of the Sequence Modelling Workshop, 2021.

[4] S. Sandoval and P. Petrova. Associative recall for linear attention models. In Proceedings of the Sequence Modelling Workshop, 2016.

[5] A. Adeyemi and X. Xu. Chunk-wise kernel for linear attention models. Transactions on Efficient Computing, 2023.